



Department of Software Engineering
University of Sialkot
Title: *Sign Language Translation System*

Submitted by:

<i>Areeba Ulfat (23010101-113)</i>
<i>Hadia Waryah (23010101-134)</i>
<i>Instructor: mam sadia</i>
<i>6th Semester (Blue)</i>

Sign Language Translation System

Abstract

This report presents the client-side architecture of the Sign Language Translation System, a Vue 3 and Vite based single page application. The system allows sign language gestures to be translated into readable text, viewed on a dashboard, searched through history, and configured through application settings. Since this update, the client no longer relies on browser storage alone: it now communicates with an Express.js and MongoDB backend through a dedicated Axios-based API service, using JWT-based authentication, while retaining localStorage for instant local caching and offline-friendly fallback. The application demonstrates Vue Router based SPA navigation, modular component design, props-based parent-to-child data flow, and custom events for child-to-parent communication.

Keywords: Vue 3, Vite, Vue Router, Components, Props, Emits, Axios, JWT Authentication, Express.js, MongoDB, Sign Language Translation.

1. Introduction

1.1 Overview

Sign language is an important communication method for many people with hearing or speech disabilities. The Sign Language Translation System is a web-based interface that simulates the structure of an AI-powered gesture recognition platform. The client is built entirely with Vue 3 and Vite, and now consumes a REST API exposed by an Express.js and MongoDB backend for authentication, history, and settings synchronisation.

The application includes public pages such as Home, About, Contact, Login, and Register. It also includes authenticated pages such as Dashboard, Translate, History, Profile, and Settings. Protected pages are guarded by a Vue Router navigation guard that checks the authentication state before allowing access.

1.2 Project Objectives

- **Frontend Framework:** Implement a complete Vue 3 frontend using Vite as the build and development tool.
- **SPA Routing:** Configure Vue Router for single page application navigation with protected and public routes.
- **Component Design:** Divide the interface into reusable, modular Vue components.
- **Props:** Demonstrate props for parent-to-child data transfer.
- **Emits:** Demonstrate custom events using defineEmits for child-to-parent communication.
- **Backend Integration:** Connect the client to an Express.js and MongoDB backend through an Axios-based API service with JWT authentication.

1.3 Why Vue 3 and Vite?

- **Cross-platform accessibility:** No installation required for end users; the application runs in any modern browser.
- **Fast development experience:** Vite provides near-instant hot module replacement during development.
- **Modular composition:** The Composition API (script setup) keeps component logic organised and reusable through composables.

2. Tools and Technologies

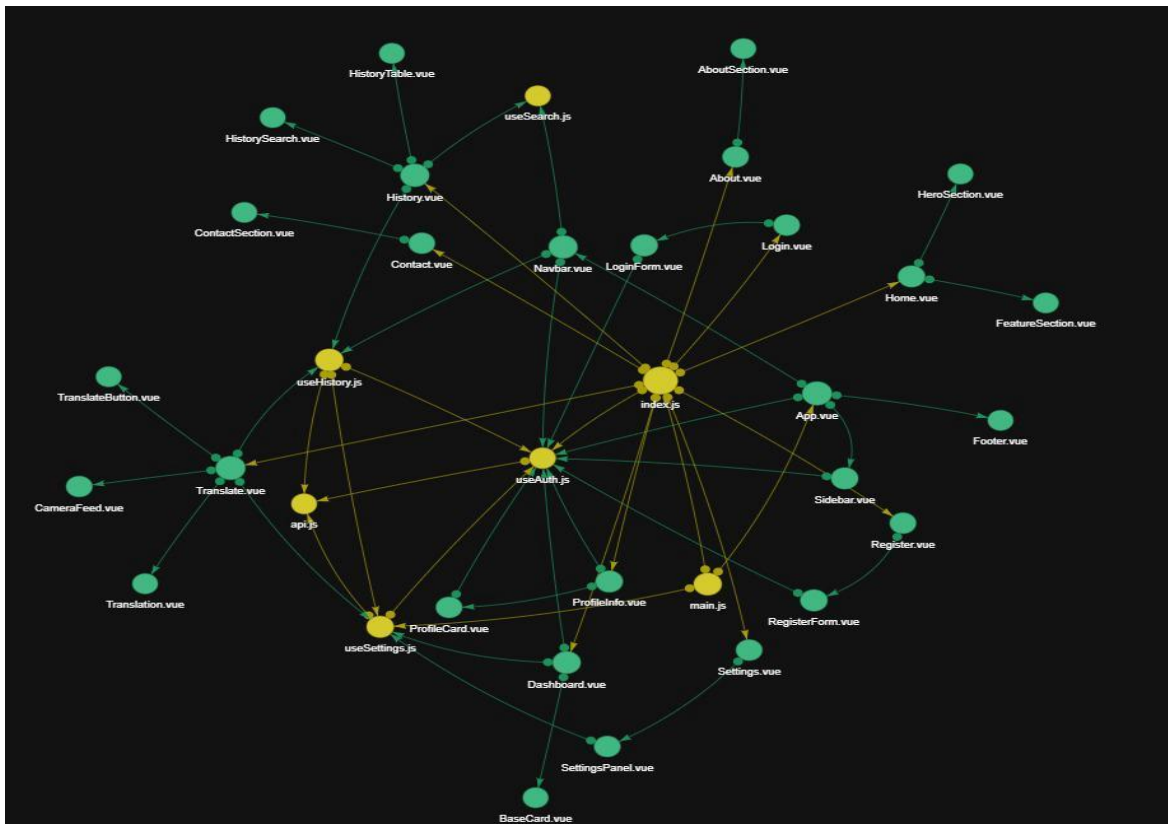
The following tools and technologies are used across the client and its backend integration:

Technology	Purpose
Vue 3	Frontend JavaScript framework (Composition API)
Vite	Development server and build tool
Vue Router	SPA routing and navigation guards
Axios	HTTP client used by src/services/api.js to call the backend
Express.js	Backend REST API server (auth, users, history)
MongoDB	Database used by the backend for persistent storage
JWT	Token-based authentication between client and backend
Bootstrap / Bootstrap Icons	Responsive styling utilities and interface icons
JavaScript	Application logic
CSS	Custom layout and styling
localStorage	Client-side caching and instant persistence fallback

3. Client-Side Architecture

The project follows a modular Vue architecture inside the client and workspace. Page-level files are stored in src and views reusable UI elements in src and components, shared frontend logic in src/composables, and backend communication in src/services.

3.1 Project Structure



3.2 Application Entry Point

The application starts from `src/main.js`. It creates the Vue application, installs the router, initialises client-side settings through `useSettings()`, loads global CSS, imports Bootstrap styling and Bootstrap Icons, and mounts the application to the HTML element with id "app".

3.3 Root Layout (App.vue)

The root component `App.vue` provides the main layout wrapper using `Navbar`, `Sidebar`, `Footer`, and `router-view`. The navbar and sidebar are shown only when the user is authenticated and is not on the Login or Register page, so those pages appear as focused full-screen forms. The layout now also supports a collapsible sidebar: the `Navbar` emits a `toggle-sidebar` event that `App.vue` listens to and uses to show or hide the `Sidebar`, which improves usability on smaller screens.

```
<template>
  <div class="app-container">
    <Navbar v-if="showShell" @toggle-sidebar="sidebarOpen = !sidebarOpen" />
    <div class="layout">
      <Sidebar v-if="showShell" v-show="sidebarOpen" />
      <main class="content">
        <router-view />
      </main>
    </div>
    <Footer />
  </div>
</template>
```

4. SPA Routing Implementation

Vue Router is configured in `src/router/index.js`. Public routes include Home, About, Login, Register, and Contact. Protected routes include Dashboard, Translate, History, Profile, and Settings (registered under both `/setting` and `/settings` for convenience).

4.1 Route Guard

A global `router.beforeEach` navigation guard checks the authentication state exposed by `useAuth()`. If a user is not authenticated and tries to open a route marked `requiresAuth`, the router redirects to Login. If an already authenticated user tries to open Login or Register, the router redirects to Dashboard instead. This satisfies the SPA routing requirement because navigation is handled entirely through Vue Router without a full page reload.

5. Modular Component Architecture

The frontend is divided into reusable components, which makes the project easier to maintain and extend.

5.1 View Components

View	Purpose
Home.vue	Landing page with hero and feature sections
Login.vue	Login page wrapper
Register.vue	Registration page wrapper
Dashboard.vue	Shows statistics, camera preview, and translation result
Translate.vue	Main translation interface
History.vue	Searchable translation history
Settings.vue	Application settings page

View	Purpose
ProfileInfo.vue	User profile page with editable profile details
About.vue	Project information page
Contact.vue	Contact page with message form

5.2 Reusable Components

Component	Purpose
Navbar.vue	Top navigation, including the sidebar toggle button
Sidebar.vue	Authenticated side navigation
Footer.vue	Footer area
BaseCard.vue	Dashboard statistic cards
CameraFeed.vue	Camera preview interface
Translation.vue	Translation result card
TranslateButton.vue	Button that emits the translate event
HistorySearch.vue	Search input for translation history
HistoryTable.vue	Displays history records
SettingsPanel.vue	User settings controls
LoginForm.vue	Login form logic and layout
RegisterForm.vue	Registration form logic and layout
ProfileCard.vue	User profile summary display
GestureCard.vue (NEW)	Displays a single gesture and its meaning
InfoCard.vue (NEW)	Generic statistic card used on informational pages

This structure follows the Vue principle of separating page-level views from reusable user interface components.

6. Props and Custom Events

The project demonstrates inter-component communication using props and custom events.

A. Props — Parent-to-Child Communication

In Translate.vue, the parent component sends translation data into the child Translation.vue component through props:

```
<Translation
  :gesture="translation.gesture"
  :translation="translation.translation"
  :confidence="translation.confidence"
  :language="translation.language"
  :status="translation.status"
  :voiceEnabled="settings.voice"
  @speak="handleSpeak"
/>
```

Inside Translation.vue these values are declared with defineProps, each with a type and default value. Another example is HistorySearch.vue, where the search query is passed from the parent view to the child input component using <HistorySearch :query="query" @search="onSearch" />.

B. Emits — Child-to-Parent Communication

The TranslateButton.vue component emits a custom translate event when the user clicks the button, and the Navbar.vue component emits a toggle-sidebar event consumed by App.vue. Both examples show correct upward communication from child components to parent views.

7. Client-Side State Management and API Integration

The project uses Vue composables for reusable state logic, kept organised without adding a heavier state library. Since this update, several composables call into a shared API service layer to synchronise data with the backend, while still falling back to localStorage.

7.1 API Service Layer (src/services/api.js) — New

This module creates a single Axios instance pointed at the backend base URL (VITE_API_URL, defaulting to `http://localhost:5000/api`). A request interceptor attaches the JWT stored under the `authToken` key to every outgoing request, and a response interceptor normalises errors so callers can keep reading `error.message` regardless of whether the failure came from the network or the backend. It exports `loginUser`, `registerUser`, `logoutUser`, `updateProfile`, `updateSettings`, `getHistory`, `saveHistoryItem`, and `deleteHistoryItem`.

7.2 Authentication State

`useAuth.js` now delegates the actual login and register calls to `services/api.js`. On success, the returned user object (`id`, `name`, `email`, and `profile` fields) is merged into local state and cached in `localStorage` so the UI still has data instantly on refresh. `logout()` clears the JWT as well as the cached authentication flags.

7.3 History State

`useHistory.js` loads history from the backend when the user is authenticated, and falls back to a `localStorage` cache otherwise or if the request fails. New items are added optimistically to the UI and then persisted to the backend through `saveHistoryItem`; removals follow the same optimistic pattern with `deleteHistoryItem`.

7.4 Settings State

`useSettings.js` applies and persists settings (theme, language, camera resolution, notifications, voice output, auto-save history, confidence threshold) to `localStorage` instantly on every change for a responsive UI, and exposes a `saveSettings()` action that explicitly syncs the current settings to the backend when the user is authenticated.

7.5 Notifications State — New

`useNotifications.js` manages an in-app notification list, tracks an unread count with a computed property, and persists notifications to `localStorage` so they survive page reloads.

8. Main Functional Pages

8. Main Functional Pages

8.1 Home Page

The Home page uses HeroSection.vue and FeatureSection.vue to introduce the system, its accuracy statistics, and its main features to visitors.

8.2 Authentication Pages

The Login and Register pages use LoginForm.vue and RegisterForm.vue. The forms validate input on the client side and then call the backend through useAuth(), which stores the returned JWT and user profile.

8.3 Dashboard Page

The Dashboard page shows system statistics through BaseCard.vue and InfoCard.vue, a camera preview through CameraFeed.vue, and a translation output card through Translation.vue.

8.4 Translate Page

The Translate page is the main functional screen. It includes a camera preview, translation result, translate button, and a live history search and table. It demonstrates props, emits, computed filtering, reactive state, and composables working together.

8.5 History Page

The History page displays saved translations retrieved from the backend (or the local cache) and allows users to search previous results using HistorySearch.vue and HistoryTable.vue.

8.6 Settings Page

The Settings page uses SettingsPanel.vue to let the user change theme, language, camera resolution, notifications, voice output, auto-save history, and confidence threshold, with an explicit action to save changes to the backend.

8.7 Profile Page

The Profile page (ProfileInfo.vue) displays and allows editing of the user's profile details, which are persisted through updateProfile() in the API service.

8.8 About Page

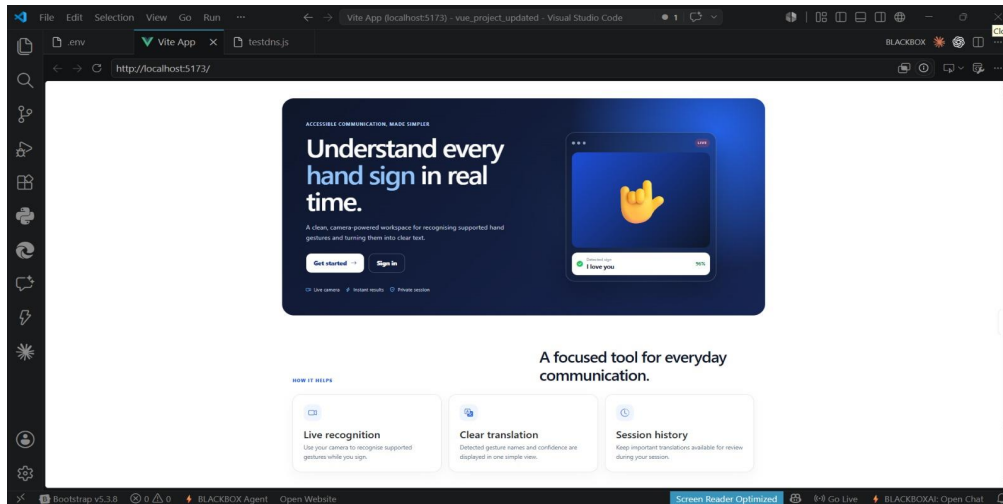
The About page uses AboutSection.vue to describe the project's mission and purpose.

8.9 Contact Page

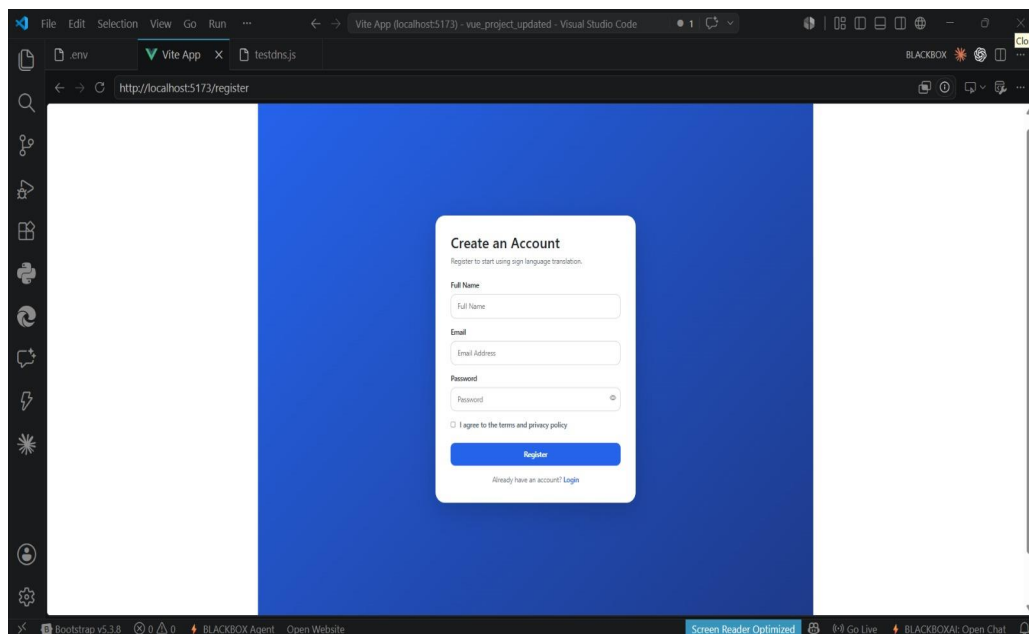
The Contact page uses ContactSection.vue with a validated message form for visitors to get in touch.

9. Screenshots

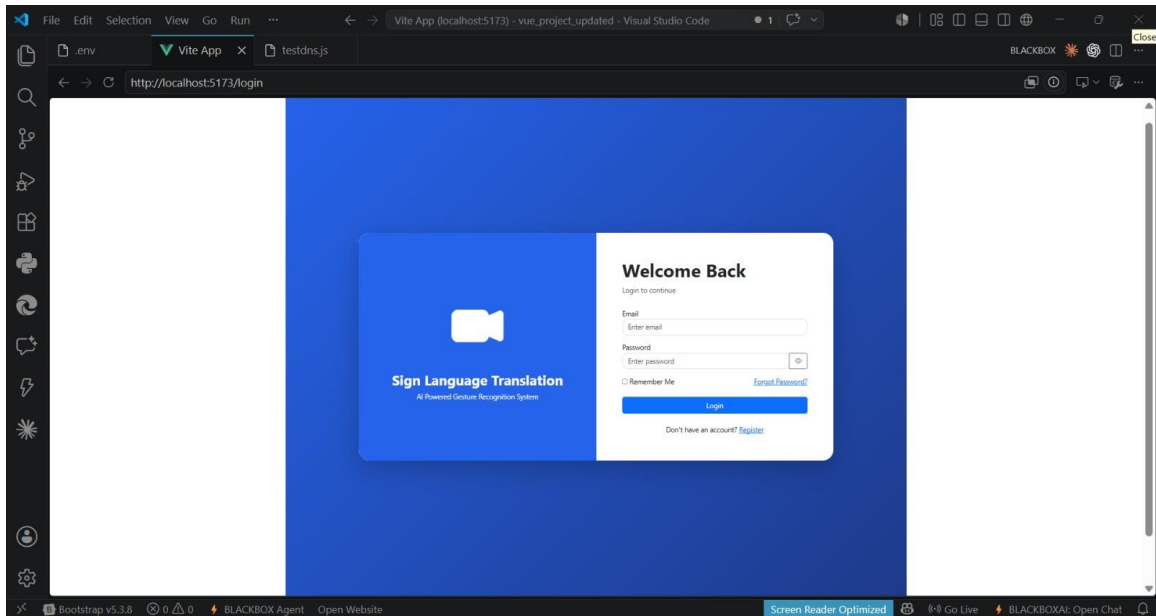
The following screenshots document the application running



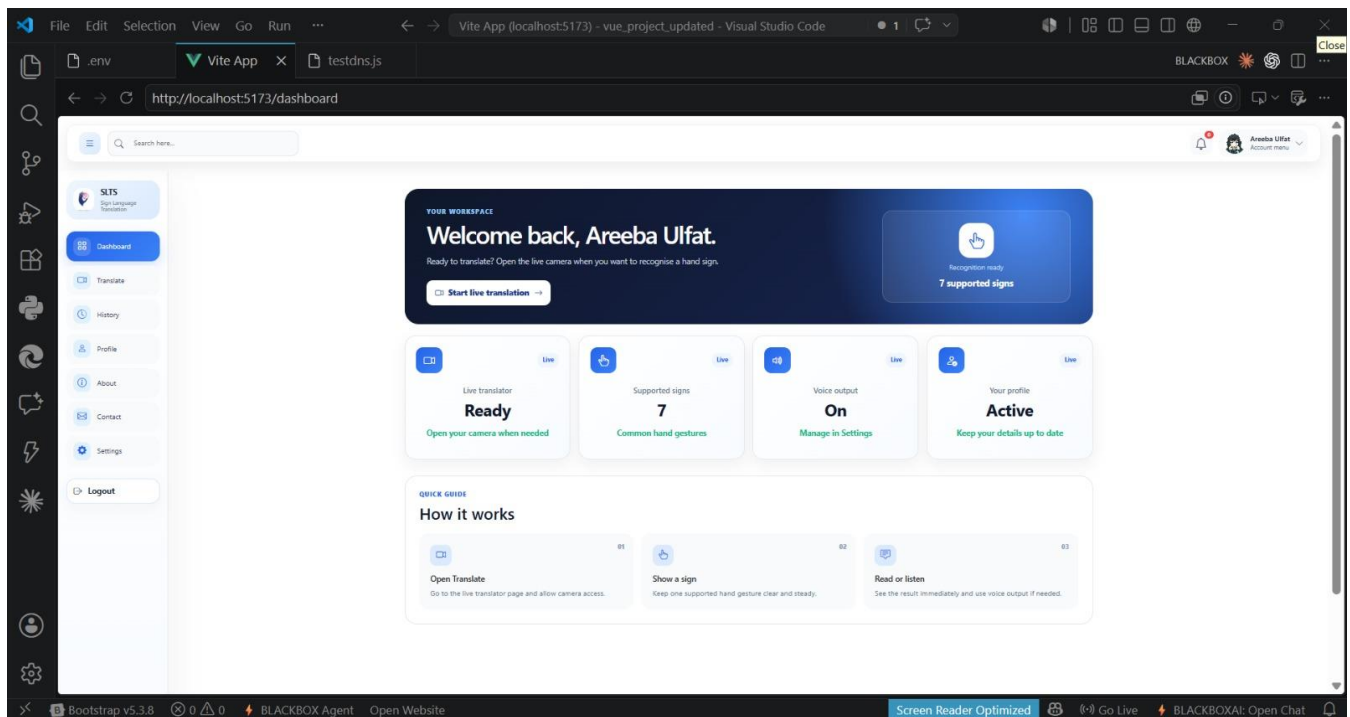
Home: The public homepage introducing the app with the tagline "Understand every hand sign in real time" a live demo preview showing a detected gesture ("I love you" – 96%) "Get started" and "Sign in" buttons and a "How it helps" section highlighting live recognition clear translation and session history features.



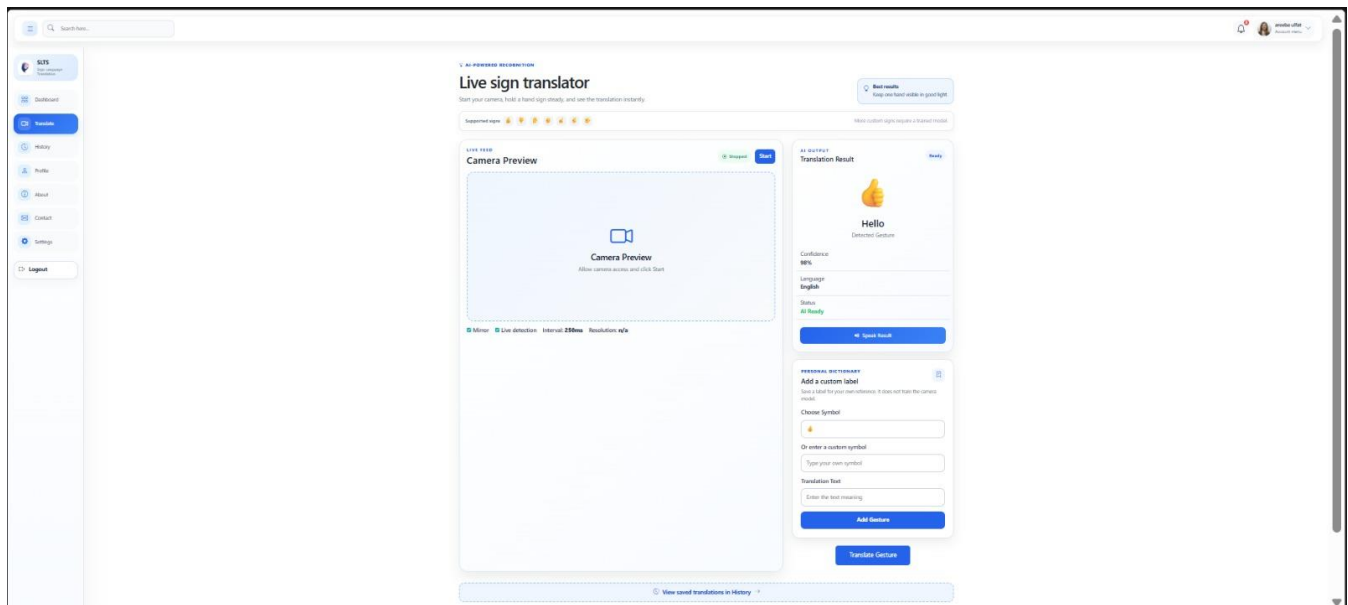
Register: Account creation page with fields for Full Name Email and Password a terms & privacy policy checkbox and a "Register" button with a link to log in if the user already has an account.



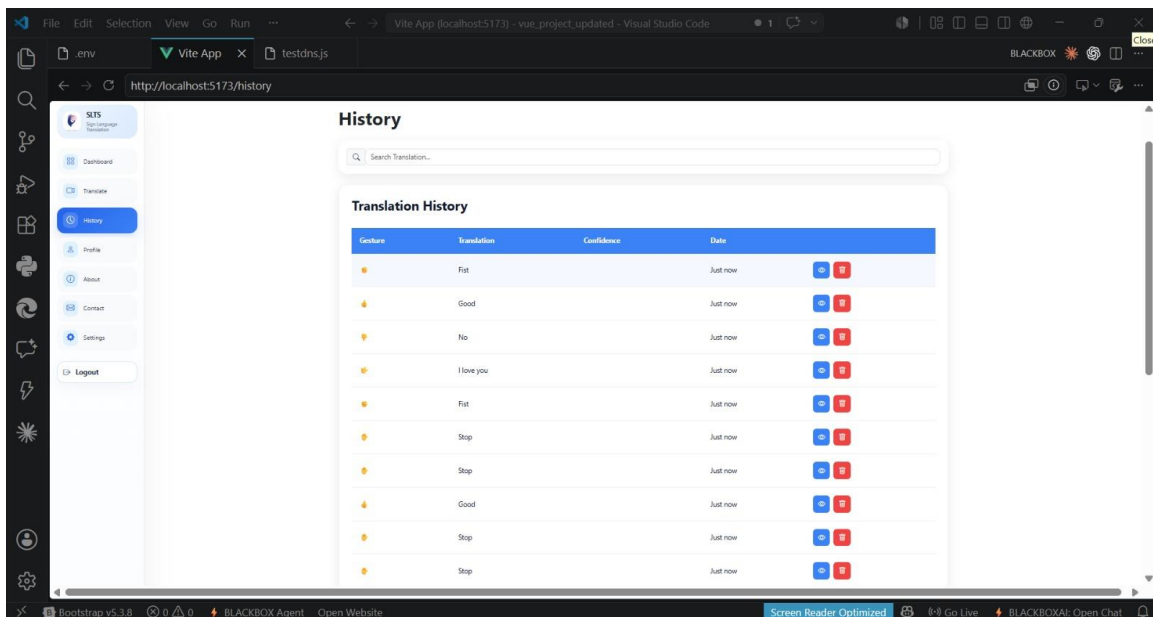
Login: Sign-in page with a branded "Sign Language Translation" panel on the left and a form on the right for Email and Password plus "Remember Me" "Forgot Password" and a link to register.



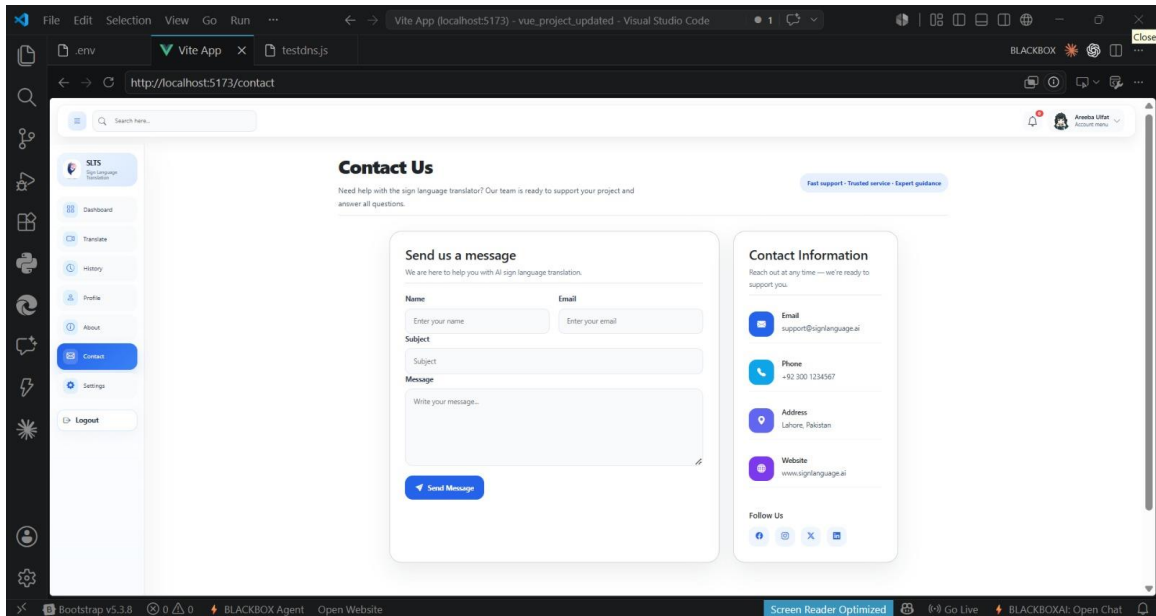
Dashboard: Welcome screen showing the user's live translator status, supported signs count (7), voice output status, and profile status. Includes a "Quick Guide" section explaining the 3-step workflow (Open Translate → Show a sign → Read or listen).



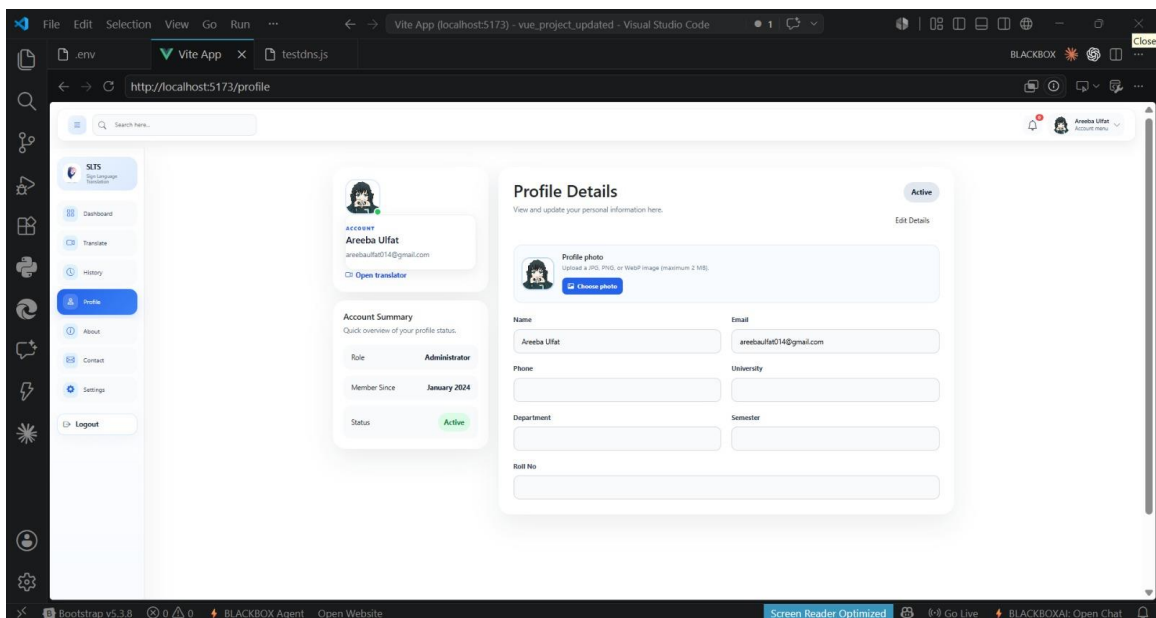
Live Sign Translator (Translate): The core translation page with a live camera preview panel for capturing hand signs and an AI Output panel showing the detected gesture ("Hello," 98% confidence) with a "Speak Result" voice button. Also includes a "Personal Dictionary" section to add custom symbol-to-text mappings and a "Translate Gesture" button to process the sign.



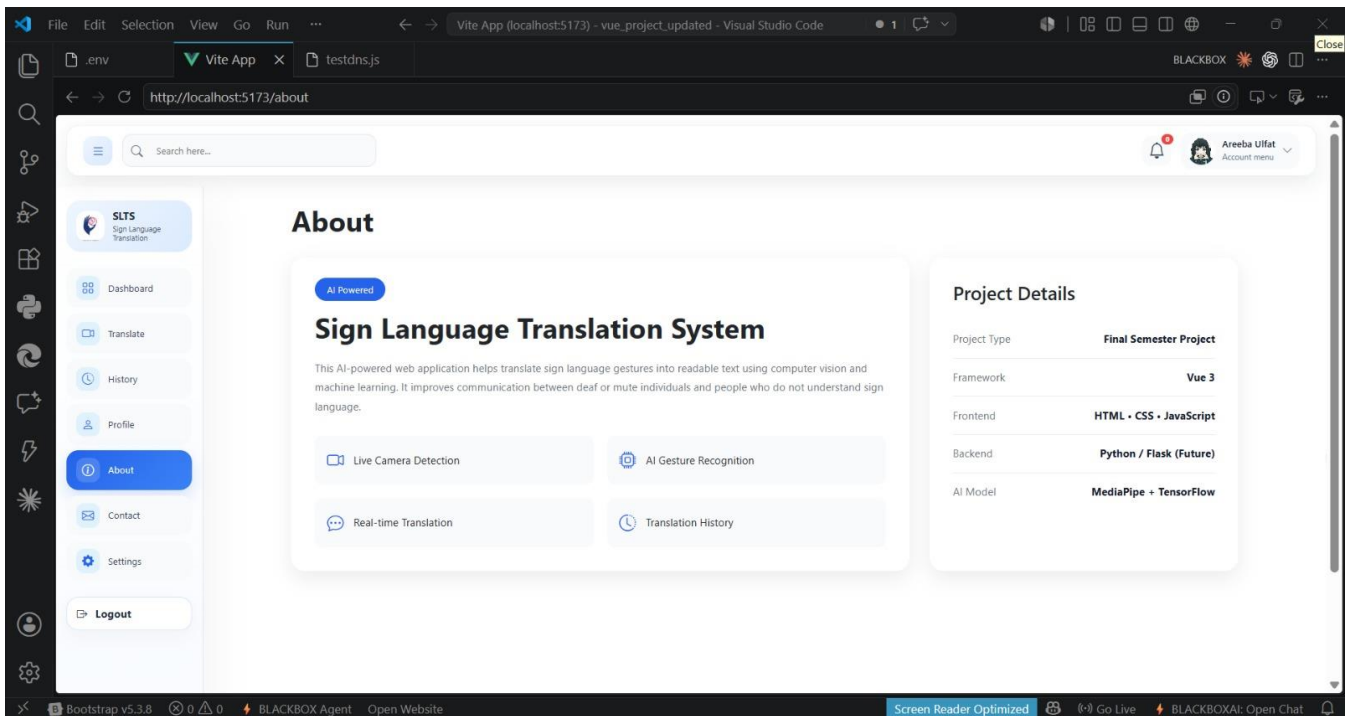
History: Displays a searchable table of past translations with columns for Gesture (icon) Translation (text) Confidence and Date along with view and delete action buttons for each entry.



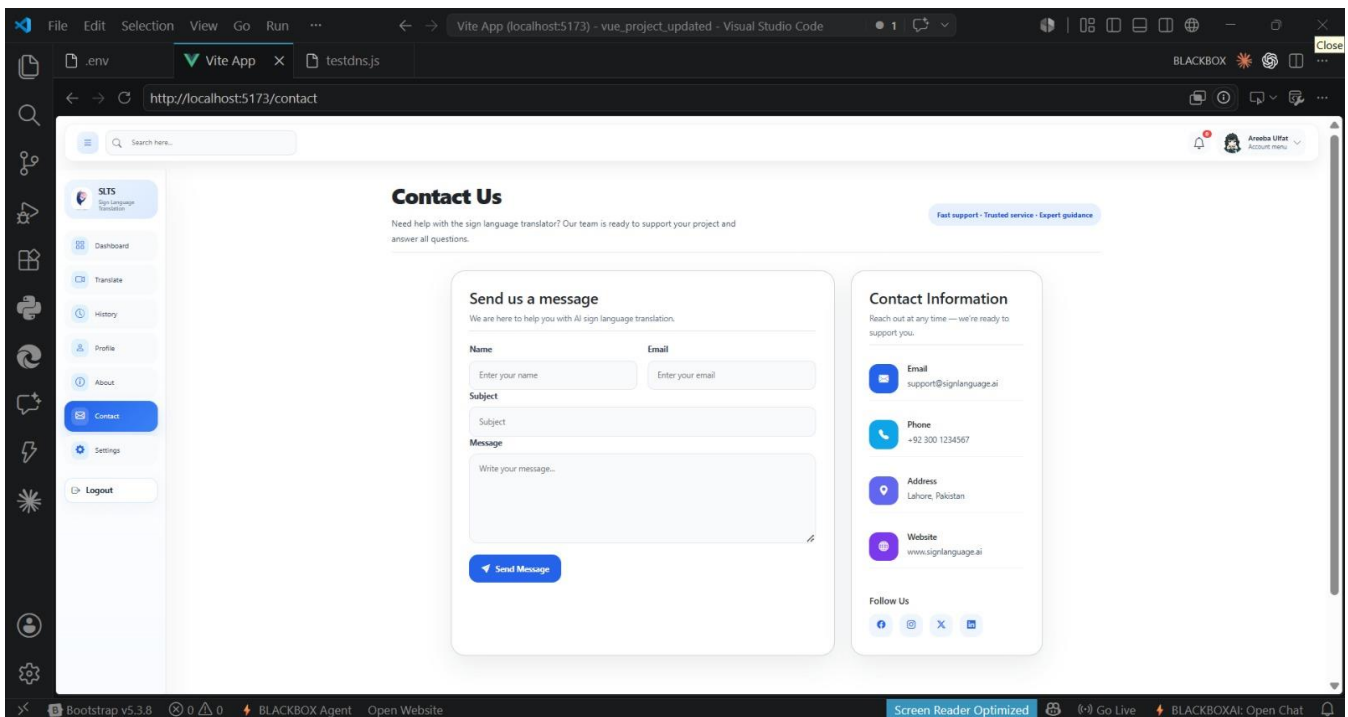
Contact: A contact form (name email subject message) alongside contact information (email phone address website) and social media links for reaching the SLTS support team.



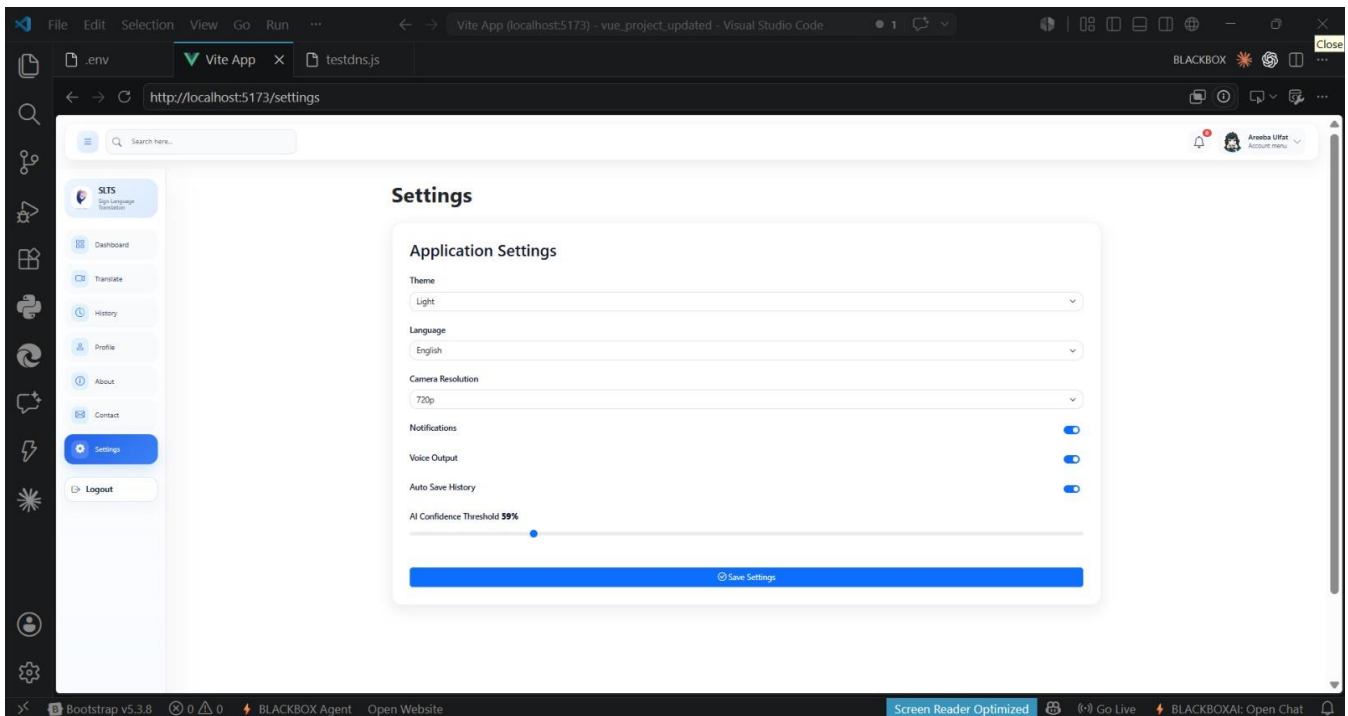
Profile: Shows user account details (name, email, role, member since, status) with an editable form for personal info like phone university department semester and roll number plus a profile photo uploader.



About: Describes the project as an AI-powered sign language translation system using computer vision and ML listing key features (Live Camera Detection AI Gesture Recognition Real-time Translation Translation History) and technical stack details (Vue 3 Python and Flask MediaPipe + TensorFlow).



Contact: A contact form (name email subject message) alongside contact information (email phone address website) and social media links for reaching the SLTS support team.



Settings: Application configuration page with options for theme (Light), language (English), camera resolution (720p), toggles for notifications/voice output/auto-save history, and an AI confidence threshold slider (currently 59%).

10. Testing and Verification

1. The application opens successfully on localhost.
2. Public routes such as Home, About, Contact, Login, and Register are accessible.
3. Protected routes redirect unauthenticated users to Login.
4. Login and Register call the backend and correctly update the authentication state and JWT.
5. The Dashboard displays statistic cards and the translation result.
6. The Translate page updates translation output from user input.
7. The Translate button emits an event to the parent component.
8. The History search filters records using a computed property.
9. The Settings page updates locally and syncs to the backend on save.
10. The sidebar toggle correctly shows and hides the Sidebar component.
11. Navigation works through RouterLink and RouterView without a full page refresh.

11. Conclusion

The Sign Language Translation System successfully implements the required client-side architecture for this deliverable. The application is built with Vue 3 and Vite, uses Vue Router for SPA navigation, and is divided into clean page and reusable component directories. It demonstrates component communication through props and custom events, uses composables for reusable frontend state, and now consumes a live Express.js and MongoDB backend through a dedicated Axios API layer with JWT authentication, while retaining localStorage for fast local caching. The frontend is ready for demonstration and further feature expansion in the next project phase.

12. References

- [1] Vue.js, "Vue.js Documentation," Vue.js, 2026. [Online]. Available: <https://vuejs.org/>
- [2] Vue Router, "Vue Router Documentation," Vue.js, 2026. [Online]. Available: <https://router.vuejs.org/>
- [3] Vite, "Vite Documentation," Vite, 2026. [Online]. Available: <https://vite.dev/>
- [4] Bootstrap, "Bootstrap Documentation," Bootstrap, 2026. [Online]. Available: <https://getbootstrap.com/>
- [5] Axios, "Axios Documentation," 2026. [Online]. Available: <https://axios-http.com/>
- [6] Express.js, "Express.js Documentation," 2026. [Online]. Available: <https://expressjs.com/>
- [7] MongoDB, "MongoDB Documentation," 2026. [Online]. Available: <https://www.mongodb.com/docs/>